



WEB APPLICATION PENETRATION TEST REPORT

Mediroza General Hospital

<https://medirozahospital.com>

Networkwalks Cybersecurity Internship — Week 3 Bonus Project

Status	AUTHORIZED
Target	https://medirozahospital.com
Testing Type	Black-box Web Application Penetration Test
Date	[25th September 2026]

⚠ This report documents an authorized security assessment conducted strictly within the defined scope and rules of engagement.

IMPORTANT — LEGAL AND ETHICAL USE ONLY

W3-BONUS | CYBERSECURITY | NETWORKWALKS**Engagement Information**

Analyst / Pentester	Safwan Abdurahiman Kavil
Role / Title	Cybersecurity Intern
Engagement Name	B083-Networkwalks — Mediroza General Hospital
Client / Target Scope	https://medirozahospital.com
Authorization on File	Yes — Approval secured from Mediroza General Hospital
Report Date	[25th September 2026]

Assessment Scope

Testing Approach	Black-box Web Application Penetration Test
Phase 1	Reconnaissance
Phase 2	Vulnerability Identification
Phase 3	Exploitation
Phase 4	Documentation

Report Index

W3-BONUS CYBERSECURITY NETWORKWALKS.....	2
Report Index.....	3
[SECTION 01] Authorization & Liability Disclaimer	4
[SECTION 02] Executive Summary	4
[SECTION 03] Scope & Objectives	5
[SECTION 04] Methodology & Tools Used	5
[SECTION 05] Activities Performed	6
5.1 Reconnaissance	6
5.2 Vulnerability Identification	7
5.3 Exploitation	7
5.4 Documentation	12
[SECTION 06] Findings & Risk Analysis	12
Web Application Risk Assessment — medirozahospital.com	12
[SECTION 07] Recommendations	14
[SECTION 08] Conclusion	16
[SECTION 09] Evidence & Appendix	17
📁 Reconnaissance	17
📁 Vulnerability Identification	21
📁 Exploitation	22
[SECTION 10] Author & Project Information	29

[SECTION 01] Authorization & Liability Disclaimer

This assessment was performed only against systems and assets for which written authorization was obtained — specifically Mediroza General Hospital's web application at <https://medirozahospital.com> — under the terms of the Networkwalks B083 Cybersecurity Internship Week 3 Bonus Project.

All activities described in this report were conducted for authorized security assessment, education and research purposes only. Nothing in this document should be used to access, scan, or test any system without explicit written permission from its owner. Every action taken is the responsibility of the person performing it. Misuse of these techniques may result in criminal charges, civil liability, loss of employment, and a permanent record. In most jurisdictions, unauthorized access to a computer system is a crime even when no damage occurs.

[SECTION 02] Executive Summary

This engagement was an authorized black-box penetration test of Mediroza General Hospital's public web application, medirozahospital.com, carried out under the Networkwalks B083 Cybersecurity Internship. Testing covered reconnaissance, vulnerability identification, and controlled exploitation, aimed at establishing what an attacker with no prior access could reach and how far that access could be pushed.

Ten findings were confirmed during the assessment: two rated Critical, three High, four Medium, and one Low. The most serious is a SQL injection flaw in the patient portal that allows an attacker to bypass the login entirely and access the system as an administrator without a valid password. That access was used to reach real patient reports, whose weak passwords were cracked, and whose metadata pointed to an exposed backup directory that was already reachable on the live site. In parallel, the hospital's server management panels (WHM, cPanel, Webmail) sit open on the internet with no IP restriction, the site is missing basic browser security headers, and outbound email carries no DMARC or DKIM protection, leaving the domain open to spoofing.

Cloudflare and the LiteSpeed firewall provide a reasonable first layer of defense, but they do not make up for these gaps at the application and configuration level. The point for leadership is straightforward: a small set of low-effort techniques, combined, gave an outside attacker full access to patient data. Given what is at stake, the Critical and High findings in Section 06 should be treated as immediate remediation priorities.

[SECTION 03] Scope & Objectives

Objectives

- Identify security weaknesses in the Mediroza General Hospital web application through black-box testing.
- Verify the real-world impact of identified weaknesses through controlled, authorized exploitation.
- Document practical, prioritized remediation steps for each confirmed finding.

In Scope

- <https://medirozahospital.com> — agreed target domain only

Out of Scope

- Social engineering (phishing, pretexting, physical access attempts)
- Denial-of-service (DoS/DDoS) testing
- Any system, subdomain, or third-party service outside the agreed target domain

Constraints / Rules of Engagement

- Testing limited strictly to the agreed target domain — no lateral testing of related infrastructure.
- No destructive testing; exploitation performed only to confirm impact, not to cause harm or downtime.
- All testing activity logged with timestamps for traceability.

[SECTION 04] Methodology & Tools Used

The assessment followed a black-box penetration testing methodology across four phases, outlined below.

Phase 1 — Reconnaissance	Gathering information about the target using publicly available sources and web-based tools (OSINT, DNS/subdomain enumeration, technology fingerprinting).
Phase 2 — Vulnerability Identification	Testing application behaviour, authentication mechanisms, session handling, and input handling for common web application weaknesses.
Phase 3 — Exploitation	Performing controlled, authorized exploitation of identified weaknesses to confirm real-world impact.
Phase 4 — Documentation	Recording findings, evidence, impact, and remediation recommendations in this report.

Tools Used

Hexstrike MCP with Claude: Used mainly for reconnaissance activities, vulnerability mapping, and integrating individual tools like whatweb, wafw00f, curl, and dig.

AI Tools (ChatGPT, Claude): Used to synthesize the findings and convert raw data into meaningful data to map vulnerabilities.

Burpsuite: To analyze web traffic and requests for any vulnerability analysis

Hash Calculation: Used pdf2john to calculate hash values of the acquired PDF files.

Password Cracking: Used NetworkWalks in-built password cracking tool to crack the passwords of the PDF files.

qpdf: Used to decrypt the PDF files using the extracted passwords for convenience.

exiftool: Used for extracting metadata of files for analysis.

[SECTION 05] Activities Performed

5.1 Reconnaissance

Infrastructure Mapped

- ▶ Target: <https://medirozahospital.com>
- ▶ Primary IP: 199.188.201.16
- ▶ Hosting Company: Namecheap (shared hosting)
- ▶ Protection Layers: Cloudflare + LiteSpeed + a load balancer
- ▶ HTTPS: Up to date.
- ▶ DNS Security: DNSSEC is OFF
- ▶ Bot detection: Available

Port Security

- ▶ 80/443: HTTP and HTTPS
- ▶ 2082/2083: Control panel for the hosting account (cPanel)
- ▶ 2086/2087: Full server management panel (WHM)
- ▶ 2095/2096: Webmail login portal

5.2 Vulnerability Identification

Authentication: WHM (2087), cPanel (2083), and Webmail (2096) login panels are externally reachable but currently gated by a Cloudflare/LiteSpeed JS.

Input Validation: A basic SQLi probe string returned an HTTP 403 (WAF block), confirming the LiteSpeed WAF is actively filtering malicious input at the edge. However, the WAF becomes pointless because the input validation on the database side has a huge vulnerability. For instance, patient portal login can be bypassed using a basic SQL injection attack with username enumeration because basic database security and password protection principles are not applied.

Access Control: Server management interfaces (WHM/cPanel/Webmail) are exposed with no IP restriction. This is a configuration-level access control gap rather than an application logic flaw.

Security Configuration: Confirmed missing security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) on the main site; SPF configured as soft-fail with no DMARC/DKIM; DNSSEC is not enabled.

Shared Hosting: Server hosts multiple sites on the same cluster. A compromise of a co-site on the same shared server could be leveraged for lateral movement.

5.3 Exploitation

AUTHORIZED EXPLOITATION SCOPE

This section explains exactly what kind of vulnerabilities were exploited according to the authorized engagement directives.

For what is authorized, this penetration test focuses on using web crawlers to identify exposed directories, acquire and analyze the data from those directories, take advantage of the input validation vulnerability in the patient portal login, and access patient records for analysis and documentation.

WEB CRAWLING & DIRECTORY DISCOVERY

Using the curl command with the /robots.txt addition, three directories were exposed on the website:

Exposed Directory	Access Status
/patient/	 Authentication Required
/staff/	 Authentication Required
/old/	 Accessible

Obviously, the /patient/ and /staff/ directories are blocked with authentication. However, the /old/ directory is accessible.

This exposed directory became significant later in the assessment after information contained within the patient reports indicated that database backups had previously been moved to this location.

INPUT VALIDATION & SQL INJECTION

Input validation was tested in both the patient and staff portals. It was discovered that the **patient portal lacks proper input validation and is vulnerable to SQL injection.**

Username Enumeration

In the patient portal, the first attempt was made by logging in with an unknown username, such as "Alice", together with a random password.

The application returned:

"Username not found"

A second attempt was then made using the username "admin" with a random password.

The application returned:

"Incorrect password"

This indicates that an **admin account exists in the patient portal.**

SQL Injection Confirmation

A third attempt was made using:

Username: admin'

Password: <any password>

The application returned the following MySQL error:

Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''' at line 1

This error demonstrates that user input is being directly processed by the SQL query and **proves that the patient portal is vulnerable to SQL injection.**

SQL INJECTION & PORTAL ACCESS

Access to the admin account was granted using the following SQL injection attempt:

Username: admin' --

Password: <any password>

The -- sign comments out the remainder of a typical SQL query.

It is highly plausible that the SQL query for the patient portal is similar to:

```
SELECT * FROM PATIENTS
WHERE username = 'admin'
AND password = '*****';
```

However, the -- operation after admin' comments out the rest of the query, resulting in:

```
SELECT * FROM PATIENTS
WHERE username = 'admin' -- ' AND password = '*****';
```

The effective query is therefore:

```
SELECT * FROM PATIENTS
WHERE username = 'admin';
```

This edited query will return the admin account without requiring the correct password, granting access to the portal.

Finding: Authentication can be bypassed through SQL injection due to insufficient input validation.

PATIENT REPORT ACCESS & PASSWORD CRACKING

Once access was granted, three password-protected patient reports were available for download.

The passwords were basic enough to be cracked using available password-cracking tools such as:

pdf2john
John the Ripper

A dictionary attack was used to recover the PDF passwords.

Once the PDF passwords were obtained, the reports could be accessed for further analysis.

PDF DECRYPTION & METADATA ANALYSIS

Since the PDFs were available, it was necessary to extract their metadata to determine whether any additional hidden or useful information was present.

Using:

qpdf
exiftool

the PDFs were decrypted and their metadata was analyzed.

One of the files, patient_report_3.pdf, contained the following metadata:

“DB backup moved to /old before site migration, do not delete”

This metadata indicates the significance of the previously discovered /old/ directory and provides a direct connection between the patient reports and the exposed directory.

ATTACK CHAINS

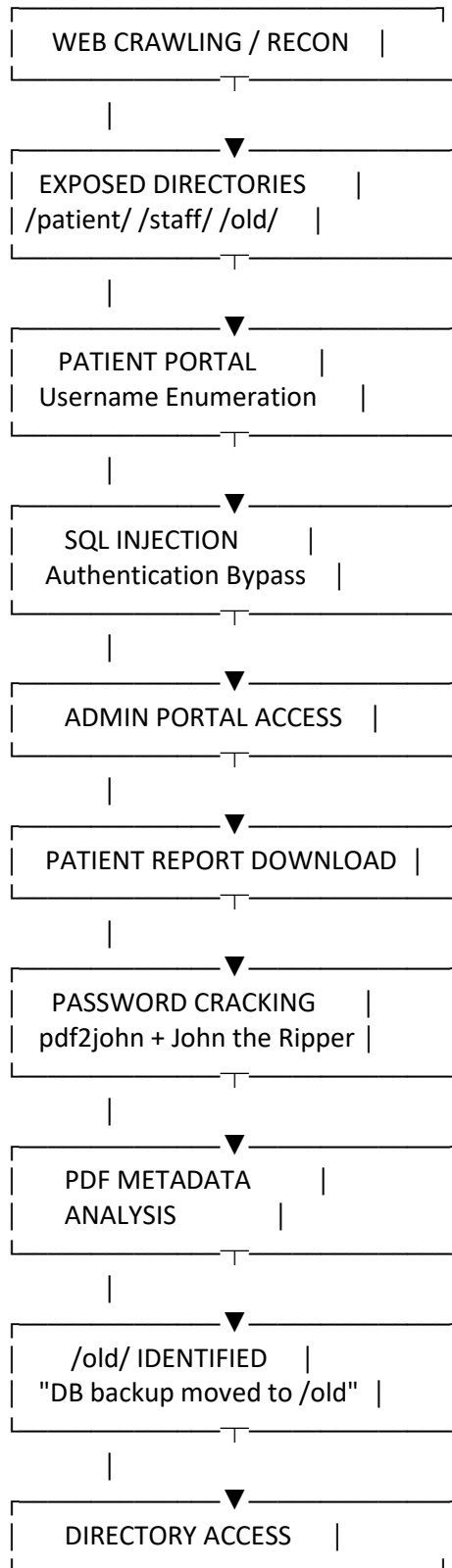
Two separate attack chains led to the /old/ directory.

ATTACK CHAIN 01: DIRECT DIRECTORY DISCOVERY

WEB CRAWLING
↓
DIRECTORY DISCOVERY
↓
/old/ IDENTIFIED
↓
DIRECTORY ACCESS

ATTACK CHAIN 02: PATIENT PORTAL COMPROMISE

RECONNAISSANCE
↓
USERNAME ENUMERATION
↓
SQL INJECTION
↓
PORTAL ACCESS
↓
PATIENT REPORT DOWNLOAD
↓
PASSWORD CRACKING
↓
METADATA ANALYSIS
↓
/old/ DIRECTORY IDENTIFIED
↓
DIRECTORY ACCESS

OVERALL ATTACK PATH

KEY OBSERVATIONS

The assessment identified two paths leading to the exposed /old/ directory:

Direct Path

Web Crawling → Detection → Directory Access

Indirect Path

Reconnaissance → Username Enumeration → SQL Injection → Portal Access → Data Exfiltration → Password Cracking → Metadata Analysis → Directory Access

The second chain demonstrates how multiple weaknesses can be combined to progress from reconnaissance and authentication weaknesses to sensitive data access and, ultimately, discovery of additional exposed resources.

5.4 Documentation

- All findings, evidence, and remediation steps consolidated into this report (Sections 6–9).

[SECTION 06] Findings & Risk Analysis

Based on the reconnaissance, vulnerability identification, and exploitation activities, the following findings were identified. Each is rated using the risk key below.

Web Application Risk Assessment — medirozahospital.com

Scope: <https://medirozahospital.com>

#	Finding	Evidence / Observation	Potential Impact	Risk
1	SQL Injection in Patient Portal	Supplying admin' generated a MySQL syntax error, confirming that user input is being incorporated into the SQL query. The payload admin' -- successfully bypassed authentication and provided access to the admin account.	An attacker could bypass authentication, gain unauthorized administrative access, access patient information, and potentially manipulate or extract sensitive database records depending on database permissions.	Critical
2	Hosting control panels exposed to the internet	WHM (2086/2087), cPanel (2082/2083), and Webmail (2095/2096) all respond as open on the public IP 199.188.201.16.	WHM is a full server-management interface — if bot-protection is bypassed or credentials are weak/leaked, an attacker gets control of the entire	Critical

#	Finding	Evidence / Observation	Potential Impact	Risk
			hosting account (and potentially co-tenant sites on the shared server).	
3	No browser-security headers on the main site	X-Frame-Options, Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy, Permissions-Policy all confirmed absent from the HTTPS response.	For a healthcare site handling patient-facing content, this leaves visitors exposed to clickjacking, XSS impact amplification, SSL-stripping, and MIME-sniffing attacks with no browser-side mitigation.	High
4	Weak email authentication	SPF record ends in ~all (soft-fail, not enforced); no DMARC or DKIM TXT records found.	Attackers can spoof @medirozahospital.com in phishing emails with a high chance of landing in inboxes which is a significant risk given the domain is healthcare-related and brand-trust-sensitive.	High
5	Exposed /old/ Directory	/old/ was accessible without authentication. PDF metadata also stated: "DB backup moved to /old before site migration, do not delete" , indicating that sensitive or historical data may be stored in this directory.	Unauthorized users may access sensitive files, database backups, legacy application data, configuration files, or other information that could further enable compromise of the environment.	High
6	Unsigned DNS	DNSSEC is unsigned.	An unsigned DNS leaves the domain open to spoofing/cache-poisoning style attacks with no cryptographic protection.	Medium
7	Shared hosting environment	Server hosts multiple sites on the same server274 cluster (confirmed via SPF-linked IPs on the same /24).	A compromise of a co-tenant site on the same shared server could be leveraged for lateral movement toward this target, outside of medirozahospital.com's own security posture.	Medium
8	Username Enumeration in Patient Portal	An unknown username returned "Username not found" , while admin returned "Incorrect password" , allowing the existence of the admin account to be identified.	Attackers can identify valid usernames and focus password attacks or other exploitation attempts against confirmed accounts,	Medium

#	Finding	Evidence / Observation	Potential Impact	Risk
			reducing the effort required for unauthorized access.	
9	Weak Password Protection on Patient Reports	Three password-protected patient reports were downloadable after portal access, and their passwords were recovered using pdf2john and John the Ripper with dictionary attacks.	Weak report passwords reduce the effectiveness of file-level protection. Anyone obtaining the PDF files may be able to recover the passwords and access sensitive patient information.	Medium
10	Sensitive Information Disclosure Through PDF Metadata	patient_report_3.pdf contained metadata stating: "DB backup moved to /old before site migration, do not delete" .	Metadata can disclose internal storage locations, migration information, backup practices, and other operational details that may assist further reconnaissance and discovery of sensitive resources.	Low

Risk level key: **Critical** | **High** | **Medium** | **Low**

These findings reflect observations from authorized reconnaissance, testing, and controlled exploitation activities. Any exploitation performed was limited to confirming impact and did not extend beyond the agreed scope.

[SECTION 07] Recommendations

Recommendations below are mapped directly to the findings in Section 06, prioritized by risk to guide remediation planning.

#	Finding Addressed	Recommendation	Priority
1	SQL Injection in Patient Portal	Implement parameterized queries or prepared statements for all database operations. Never concatenate user-supplied input directly into SQL statements. Validate and sanitize input on the server side, use secure authentication mechanisms, and suppress detailed database error messages from being displayed to users.	Critical

#	Finding Addressed	Recommendation	Priority
2	Exposed WHM/cPanel/Webmail panels	Restrict ports 2082/2083/2086/2087/2095/2096 to an IP allow-list (admin/office IPs) or place behind a VPN; if not achievable at the hosting level, request Namecheap enable IP-based ACLs on these panels.	Critical
3	Missing security headers	Add Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy at the web-server/CDN (Cloudflare) config level — low-effort, no downtime.	High
4	Weak SPF, missing DMARC/DKIM	Change SPF to -all (hard fail) once all legitimate senders are confirmed; publish a DMARC record (p=quarantine initially, moving to p=reject) and enable DKIM signing on outbound mail.	High
5	Exposed /old/ Directory	Remove the /old/ directory from public web access if it is no longer required. If the directory must be retained, restrict access through authentication and appropriate server-side access controls. Sensitive backups, database dumps, configuration files, and legacy data should be stored outside the web-accessible document root.	High
6	Unsigned DNS	Enable DNSSEC with the current registrar (NameCheap supports this natively).	Medium
7	Shared hosting exposure	No direct fix from the target's side, but document as an accepted risk; consider migrating to an isolated VPS/dedicated environment if the site will handle sensitive patient data.	Medium
8	Username Enumeration in Patient Portal	Use a generic authentication error such as " Invalid username or password " for both invalid usernames and invalid passwords. Avoid revealing whether an account exists and consider rate limiting or account lockout protections against repeated authentication attempts.	Medium
9	Weak Password Protection on Patient Reports	Enforce strong, randomly generated passwords for protected patient reports. Avoid predictable or dictionary-based passwords and use sufficiently long passphrases or randomly generated secrets. Where possible, protect sensitive documents through authenticated access rather than relying solely on PDF passwords.	Medium
10	Sensitive Information Disclosure Through PDF Metadata	Remove unnecessary metadata from documents before publication or download. Metadata should not contain internal paths, backup locations, migration notes, usernames, system information, or other operational details that could assist reconnaissance.	Low

Priority level key: Critical | High | Medium | Low

[SECTION 08] Conclusion

This assessment combined reconnaissance, vulnerability identification, and controlled exploitation to evaluate the real security of Mediroza General Hospital's web application at medirozahospital.com. Reconnaissance mapped the hosting environment and surfaced several internet-facing management panels along with DNS and email misconfigurations. Vulnerability identification confirmed these were not isolated problems: the patient portal's input validation was broken at the database layer, standard browser security headers were missing, and the LiteSpeed firewall blocking a basic injection attempt did nothing to stop attacks against the underlying database logic itself.

Exploitation showed how these gaps play out in practice. Two separate paths led to the same exposed `/old/` directory, one through simple web crawling, and a second through username enumeration and SQL injection that bypassed authentication entirely, reached real patient reports, cracked their passwords, and used their metadata to confirm the location of an old database backup. Neither path required advanced tooling or specialist skill. That is the central lesson of this engagement: no single finding here needed to be severe on its own for the overall impact to be serious, because a handful of ordinary weaknesses, chained together, produced full unauthorized access to sensitive patient information.

All testing and exploitation described in this report was carried out strictly within the scope authorized by Mediroza General Hospital, with no destructive activity and nothing tested outside medirozahospital.com. The findings and recommendations in Sections 06 and 07 should be actioned promptly, prioritizing the Critical items, to close the gap between the hospital's current security posture and the protection patient data of this sensitivity requires.

[SECTION 09] Evidence & Appendix

Reconnaissance

Screenshot

```
(kali㉿kali)-[~/Downloads]
$ dig A medirozahospital.com +short
199.188.201.16

(kali㉿kali)-[~/Downloads]
$ dig MX medirozahospital.com +short
5 mx1-hosting.jellyfish.systems.
20 mx3-hosting.jellyfish.systems.
10 mx2-hosting.jellyfish.systems.

(kali㉿kali)-[~/Downloads]
$ dig NS medirozahospital.com +short
dns1.namecheaphosting.com.
dns2.namecheaphosting.com.

(kali㉿kali)-[~/Downloads]
$ dig TXT medirozahospital.com +short
"v=spf1 +a +mx +ip4:199.188.201.7 +ip4:199.188.201.38 include:spf.web-hosting.com ~all"

(kali㉿kali)-[~/Downloads]
$ whois medirozahospital.com
Domain Name: MEDIROZAHOSPITAL.COM
Registry Domain ID: 3132326963_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.namecheap.com
Registrar URL: http://www.namecheap.com
Updated Date: 2026-08-14T21:07:45Z
Creation Date: 2026-08-14T20:51:24Z
Registry Expiry Date: 2027-08-14T20:51:24Z
Registrar: NameCheap, Inc.
Registrar IANA ID: 1068
Registrar Abuse Contact Email: abuse@namecheap.com
Registrar Abuse Contact Phone: +1.6613102107
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: DNS1.NAMECHEAPHOSTING.COM
Name Server: DNS2.NAMECHEAPHOSTING.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-09-25T17:12:50Z <<<

For more information on Whois status codes, please visit https://icann.org/epp
```

Screenshot

```
(kali@kali)-[~/Downloads]
$ curl -sI https://medirozahospital.com
HTTP/2 200
content-type: text/html
last-modified: Fri, 04 Sep 2026 02:03:12 GMT
accept-ranges: bytes
content-length: 5445
date: Fri, 25 Sep 2026 17:13:40 GMT
server: LiteSpeed
x-turbo-charged-by: LiteSpeed

(kali@kali)-[~/Downloads]
$ curl -sI http://medirozahospital.com
HTTP/1.1 301 Moved Permanently
keep-alive: timeout=5, max=100
date: Fri, 25 Sep 2026 17:13:48 GMT
server: LiteSpeed
location: https://medirozahospital.com/
x-turbo-charged-by: LiteSpeed

(kali@kali)-[~/Downloads]
$ curl -sv https://medirozahospital.com/
* Trying 199.188.201.16:443 ...
* Host medirozahospital.com:443 was resolved.
* IPv6: (none)
* IPv4: 199.188.201.16
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust Anchors:
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / x25519 / RSASSA-PSS
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=medirozahospital.com
*  start date: Aug 14 20:59:46 2026 GMT
```

```
(kali@kali)-[~/Desktop/mediroza/recon]
$ theHarvester -d medirozahospital.com -b all
Read proxies.yaml from /etc/theHarvester/proxies.yaml
*****
*
* theHarvester
*
* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*
*****
[*] Target: medirozahospital.com
```

Screenshot

```
[*] LinkedIn Links found: 0
```

```
[*] IPs found: 2
```

```
199.188.201.16
```

```
[*] Emails found: 1
```

```
info@medirozahospital.com
```

```
[*] No people found.
```

```
[*] Hosts found: 2
```

```
ftp.medirozahospital.com  
mail.medirozahospital.com
```

```
# robots.txt
```

```
User-agent: *
```

```
Disallow: /patient/
```

```
Disallow: /staff/
```

```
Disallow: /old/
```

```
Sitemap: https://medirozahospital.com/sitemap.xml
```

```
(kali㉿kali)-[~/Downloads]
```

```
$ curl https://medirozahospital.com/robots.txt
```

```
# robots.txt
```

```
User-agent: *
```

```
Disallow: /patient/
```

```
Disallow: /staff/
```

```
Disallow: /old/
```

```
Sitemap: https://medirozahospital.com/sitemap.xml
```

```
(kali㉿kali)-[~/Downloads]
```

```
$
```

Screenshot

```

(kali@kali)-[~/Downloads]
$ wafw00f https://medirozahospital.com -a

```



404 Hack Not Found

405 Not Allowed

403 Forbidden

502 Bad Gateway

500 Internal Error

~ WAFW00F : v2.4.2 ~
The Web Application Firewall Fingerprinting Toolkit

```

[*] Checking https://medirozahospital.com
[+] The site https://medirozahospital.com is behind LiteSpeed (LiteSpeed Technologies) WAF.
[+] Generic Detection results:
[*] The site https://medirozahospital.com seems to be behind a WAF or some sort of security solution
[~] Reason: The server returns a different response code when an attack string is used.
Normal response code is "200", while the response code to a SQL injection attack is "403"
[~] Number of requests: 7

```

```

(kali@kali)-[~/Downloads]
$ whatweb -v https://medirozahospital.com
WhatWeb report for https://medirozahospital.com
Status      : 200 OK
Title       : One moment, please ...
IP          : 199.188.201.16
Country     : UNITED STATES, US

Summary     : HTML5, HTTPServer[openresty/1.31.1.1], OpenResty[1.31.1.1], Script, UncommonHeaders[cf-edge-cache]

Detected Plugins:
[ HTML5 ]
    HTML version 5, detected by the doctype declaration

[ HTTPServer ]
    HTTP server header string. This plugin also attempts to
    identify the operating system from the server header.

    String      : openresty/1.31.1.1 (from server string)

[ OpenResty ]
    OpenResty® is a dynamic web platform based on NGINX and
    LuaJIT.

    Version     : 1.31.1.1
    Google Dorks: (1)
    Website     : https://openresty.org/

[ Script ]
    This plugin detects instances of script HTML elements and
    returns the script language/type.

[ UncommonHeaders ]
    Uncommon HTTP server headers. The blacklist includes all
    the standard headers and many non standard but common ones.
    Interesting but fairly common headers should have their own
    plugins, eg. x-powered-by, server and x-aspnet-version.
    Info about headers can be found at www.http-stats.com

    String      : cf-edge-cache (from headers)

HTTP Headers:
HTTP/1.1 200 OK
Server: openresty/1.31.1.1
Date: Fri, 25 Sep 2026 17:16:43 GMT
Content-Length: 12076
Connection: close
Content-Type: text/html
Cache-Control: private, no-cache, no-store, must-revalidate, max-age=0
cf-edge-cache: no-cache

```

Vulnerability Identification

Screenshot

Reconnaissance Summary — medirozahospital.com

All 6 phases completed within scope. Here are the headline findings:

High-Priority Findings

#	Finding	Detail
1	WHM Panel Exposed	Port 2087 (HTTPS) externally accessible — hosting management for the entire server
2	cPanel Exposed	Ports 2082/2083 publicly reachable — full hosting control panel
3	Zero Security Headers	No HSTS, CSP, X-Frame-Options, X-Content-Type-Options, or Referrer-Policy on a healthcare site
4	No DMARC / DKIM	Combined with SPF -all softfail — email spoofing/impersonation is viable

Notable Infrastructure Details

- IP: **199.188.201.16** — Namecheap shared hosting (**server274-1.web-hosting.com**)
- Stack: OpenResty 1.31.1.1 (NGINX/LuaJIT) behind Cloudflare edge + LiteSpeed WAF
- TLS: TLS 1.3, AES-256-GCM, X25519MLKEM768 (post-quantum hybrid) — strong
- Bot Protection: JS browser-fingerprint challenge intercepts all automated tools — actual CMS/backend not yet fingerprinted
- Domain Age: Only registered **2026-08-14** (~40 days old)
- Webmail: Ports 2095/2096 also externally exposed

Scope Note on Path Enumeration

Every path check (**`/.env`**, **`/.git/HEAD`**, **`/wp-login.php`**, etc.) returned the JS challenge page (HTTP 200 with bot-check content) — **not real 200s**. Actual path enumeration requires a browser session to pass the Cloudflare/OpenResty challenge first. This is recommended for Phase 2.

```
(kali@kali)-[~/Downloads]
$ curl https://medirozahospital.com/robots.txt
# robots.txt
User-agent: *
Disallow: /patient/
Disallow: /staff/
Disallow: /old/
Sitemap: https://medirozahospital.com/sitemap.xml
```

```
(kali@kali)-[~/Downloads]
$
```

Screenshot

Patient Portal

Sign in to view your lab reports and results.

Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "" at line 1

Username

admin'

Password

☐ Show password

Sign in

 Exploitation

Screenshot

Patient Portal

Sign in to view your lab reports and results.

Username not found

Username

Password

☐ Show password

Sign in

Screenshot

Patient Portal

Sign in to view your lab reports and results.

Incorrect password

Username

admin

Password

☐ Show password

Sign in

Patient Portal

Sign in to view your lab reports and results.

Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1

Username

admin'

Password

☐ Show password

Sign in

Screenshot

Patient Portal

Sign in to view your lab reports and results.

Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "" at line 1

Username

admin' --

Password

••••••••

☐ Show password

Sign in



[Home](#) [Log out](#)

My lab reports

Your reports are password protected. Use the password sent to you by the laboratory to open each file.

Pathology Report - S. Dlamini

Lab Ref LR-2024-1187 | 2024-11-04 | PDF (encrypted)

Download

Pathology Report - P. Reddy

Lab Ref LR-2024-1192 | 2024-11-05 | PDF (encrypted)

Download

Pathology Report - E. Thompson

Lab Ref LR-2024-1205 | 2024-11-06 | PDF (encrypted)

Download

Screenshot

```
(kali㉿kali)-[~/Downloads]
$ /usr/share/john/pdff2john.pl patient_report_1.pdf > pdfhash1.txt

(kali㉿kali)-[~/Downloads]
$ /usr/share/john/pdff2john.pl patient_report_2.pdf > pdfhash2.txt

(kali㉿kali)-[~/Downloads]
$ /usr/share/john/pdff2john.pl patient_report_3.pdf > pdfhash3.txt
```

The screenshot shows the 'Password Cracker through Dictionary Attacks' web application. The interface is dark-themed with a purple and blue color scheme. At the top, there's a logo for 'Network Walks' and the title 'Password Cracker through Dictionary Attacks'. Below the title, a brief description states: 'Hash every word in a wordlist and match it against a PDF password hash, the same idea John the Ripper uses.' There are three numbered instructions: 1. Extract the \$pdf\$ hash in the Hash Calculator, 2. Paste it below, 3. Run the attack, then open your PDF with the password. The main area shows the progress of the attack: 'Tried: 1 / 100', '0 pw/s', and '1%'. A log of the attack process is displayed, showing the loaded wordlist, target information, and the successful match of the password '123456'. Below the log, a large green box displays the cracked password '123456' with the text 'PASSWORD CRACKED SUCCESSFULLY' and 'This is the PDF password. Open your PDF and enter it to unlock the file.' A 'Copy password' button is present. At the bottom, there's a section for the PDF hash (\$PDF\$...), showing a long alphanumeric string. Below this, there are buttons for 'Built-in list (100 passwords)', 'Upload wordlist (.txt)', and a status indicator 'Active: built-in list, 100 words'. A large 'START CRACKING' button is at the very bottom.

Network Walks

Dictionary Attack Lab

Password Cracker through Dictionary Attacks

Hash every word in a wordlist and match it against a PDF password hash, the same idea John the Ripper uses.

- 1 Extract the \$pdf\$ hash in the [Hash Calculator](#)
- 2 Paste it below
- 3 Run the attack, then open your PDF with the password

Tried: 1 / 100 0 pw/s 1%

[*] Loaded wordlist: built-in list (100 entries)
[*] Target: PDF R3 / 128-bit / dictionary attack
[*] Deriving keys with MD5 + RC4 ...
[+] MATCH 123456 ✓

PASSWORD CRACKED SUCCESSFULLY

123456

This is the PDF password. Open your PDF and enter it to unlock the file.

[Copy password](#)

PDF HASH (\$PDF\$...)

\$pdf\$2*3*128*4294967292*1*32*3361663365326235643333353531613238303137316238333238373763353339*32*ef16c52ab8efce2c18c79e9d28895b5928bf4e5e4e758a4164004e56ffa0108*32*c431fab9cc5ef7b59c244b61b745f71ac5ba427b1b9102da468e77127f1e69d6

Built-in list (100 passwords) Upload wordlist (.txt) Active: built-in list, 100 words

START CRACKING

Screenshot

The screenshot shows the 'Password Cracker through Dictionary Attacks' web application. The interface is dark-themed with a purple and blue color scheme. At the top, there's a header with the 'Network Walks' logo and the title 'Password Cracker through Dictionary Attacks'. Below the title, a subtitle explains the concept: 'Hash every word in a wordlist and match it against a PDF password hash, the same idea John the Ripper uses.' A three-step instruction list is provided: 1. Extract the \$pdf\$ hash in the Hash Calculator, 2. Paste it below, 3. Run the attack, then open your PDF with the password. The main interface features a progress bar at the top showing 'Tried: 2 / 100' and '0 pw/s' with a '2%' completion rate. Below this, a terminal window displays the following output: '[*] Loaded wordlist: built-in list (100 entries)', '[*] Target: PDF R3 / 128-bit / dictionary attack', '[*] Deriving keys with MD5 + RC4 ...', '[-] Trying: 123456 X', and '[+] MATCH password ✓'. A large green box in the center displays 'PASSWORD CRACKED SUCCESSFULLY' followed by the word 'password' in large green letters. Below this, a message states 'This is the PDF password. Open your PDF and enter it to unlock the file.' with a 'Copy password' button. At the bottom, there's a section for 'PDF HASH (\$PDF\$...)' showing a long alphanumeric hash. Below the hash, there are two buttons: 'Built-in list (100 passwords)' and 'Upload wordlist (.txt)'. To the right, a status indicator shows 'Active: built-in list, 100 words'. At the very bottom, a large blue button labeled 'START CRACKING' is visible.

Network Walks

DICTIONARY ATTACK LAB

Password Cracker through Dictionary Attacks

Hash every word in a wordlist and match it against a PDF password hash, the same idea John the Ripper uses.

- 1 Extract the \$pdf\$ hash in the [Hash Calculator](#)
- 2 Paste it below
- 3 Run the attack, then open your PDF with the password

Tried: 2 / 100 0 pw/s 2%

```
[*] Loaded wordlist: built-in list (100 entries)
[*] Target: PDF R3 / 128-bit / dictionary attack
[*] Deriving keys with MD5 + RC4 ...
[-] Trying: 123456 X
[+] MATCH password ✓
```

PASSWORD CRACKED SUCCESSFULLY

password

This is the PDF password. Open your PDF and enter it to unlock the file.

[Copy password](#)

PDF HASH (\$PDF\$...)

```
$pdf$2*3*128*4294967292*1*32*3166346338373236356437626464363834663737303265633666363264616463*32*39f4e6b0a
edf12344c340ffb39f8905528bf4e5e4e758a4164004e56fffa0108*32*408b37bcf12da873d7f2840f3c1b917a023961ded4c8164
d38e46e9655e66775
```

[Built-in list \(100 passwords\)](#) [Upload wordlist \(.txt\)](#) [Active: built-in list, 100 words](#)

START CRACKING

Screenshot



DICTIONARY ATTACK LAB

Password Cracker through Dictionary Attacks

Hash every word in a wordlist and match it against a PDF password hash, the same idea John the Ripper uses.

1

 Extract the \$pdf\$ hash in the [Hash Calculator](#)

2

 Paste it below

3

 Run the attack, then open your PDF with the password

Tried: 3546 / 3556

16 pw/s

100%

```
[~] Trying: stivers X
[~] Trying: tapani X
[~] Trying: targas X
[~] Trying: test2 X
[~] Trying: test3 X
[~] Trying: tula X
[~] Trying: unix X
[~] Trying: user1 X
[~] Trying: xanth X
[+] MATCH !@#$$%^& ✓
```

PASSWORD CRACKED SUCCESSFULLY

!@#\$\$%^&

This is the PDF password. Open your PDF and enter it to unlock the file.

Copy password

PDF HASH (\$PDF\$...)

\$pdf\$2*3*128*4294967292*1*32*3261393066326130336634386337323631306164373264323130316137616538*32*5090fa0a5dba99cb97c9d140cd23119428bf4e5e4e758a4164004e56ffa0108*32*58e03d692cf37b50b0b5eaa189fcbd372260a949c8992ad7b44fd13e2b40c1f8

Built-in list (100 passwords)

Upload wordlist (.txt)

Active: JTR_default_password.txt, 3556 words

START CRACKING

```

(kali@kali)-[~/Downloads]
$ qpdf --password='!@#$$%^&' --decrypt patient_report_3.pdf report3.pdf

(kali@kali)-[~/Downloads]
$ qpdf --password='password' --decrypt patient_report_2.pdf report2.pdf

(kali@kali)-[~/Downloads]
$ qpdf --password='123456' --decrypt patient_report_1.pdf report1.pdf

(kali@kali)-[~/Downloads]
$ exiftool report3.pdf
ExifTool Version Number      : 13.55
File Name                    : report3.pdf
Directory                    : .
File Size                    : 2.9 kB
File Modification Date/Time  : 2026:09:25 11:45:30-04:00
File Access Date/Time       : 2026:09:25 11:45:30-04:00
File Inode Change Date/Time  : 2026:09:25 11:45:30-04:00
File Permissions             : -rw-rw-r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.4
Linearized                   : No
Author                       : j.malik
Comments                     : DB backup moved to /old before site migration, do not delete
Creator                      : Mediroza CMS 1.4.2
Keywords                     : pathology, patient report
Producer                     : Mediroza Lab Reporting Module
Subject                      : Full Blood Count
Title                        : Pathology Report - E. Thompson
Page Count                   : 1

(kali@kali)-[~/Downloads]
$ exiftool report1.pdf
ExifTool Version Number      : 13.55
File Name                    : report1.pdf
Directory                    : .
File Size                    : 2.8 kB
File Modification Date/Time  : 2026:09:25 11:46:07-04:00
File Access Date/Time       : 2026:09:25 11:46:07-04:00
File Inode Change Date/Time  : 2026:09:25 11:46:07-04:00
File Permissions             : -rw-rw-r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.4
Linearized                   : No
Author                       : Mediroza Diagnostics Lab
Creator                      : Mediroza CMS 1.4.2
Keywords                     : pathology, fbc, patient report
Producer                     : Mediroza Lab Reporting Module
Subject                      : Full Blood Count
Title                        : Pathology Report - S. Dlamini
Page Count                   : 1

(kali@kali)-[~/Downloads]
$ exiftool report2.pdf
ExifTool Version Number      : 13.55
File Name                    : report2.pdf
Directory                    : .
File Size                    : 2.8 kB
File Modification Date/Time  : 2026:09:25 11:45:50-04:00
File Access Date/Time       : 2026:09:25 11:45:50-04:00
File Inode Change Date/Time  : 2026:09:25 11:45:50-04:00
File Permissions             : -rw-rw-r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.4
Linearized                   : No
Author                       : Mediroza Diagnostics Lab
Creator                      : Mediroza CMS 1.4.2
Keywords                     : pathology, lipids, patient report
Producer                     : Mediroza Lab Reporting Module
Subject                      : Lipid Profile
Title                        : Pathology Report - P. Reddy
Page Count                   : 1

(kali@kali)-[~/Downloads]
$

```

[SECTION 10] Author & Project Information

Author

Safwan Abdurahiman Kavil — CompTIA Security+

LinkedIn: www.linkedin.com/in/safwan-abdurahiman-kavil-sak03

Project Information

Program: W3-BONUS | Week: 3 (Bonus Project) | Client: Mediroza General Hospital